Android
Unidad 1. Introducción

PROGRAMACIÓN MULTIMEDIA Y DE DISPOSITIVOS MÓVILES

2º DAM

Índice

1. Android
    1. Arquitectura
    2. Aplicación
    3. Actividad
    4. *Android Manifest*
2. Android Studio
    1. Crear nuevo proyecto
    2. IU
    3. Ficheros del proyecto
    4. Ejecutar proyecto
    5. Importar proyecto
    6. *Gradle*
    7. *Debugger*
    8. Consola *Logcat*

Android: Arquitectura

El *Framework* de *Android* ofrece muchas clases útiles para ayudarnos a interactuar con el sistema o a acceder a algún tipo de funcionalidad del dispositivo, como la cámara, wifi o el Bluetooth.

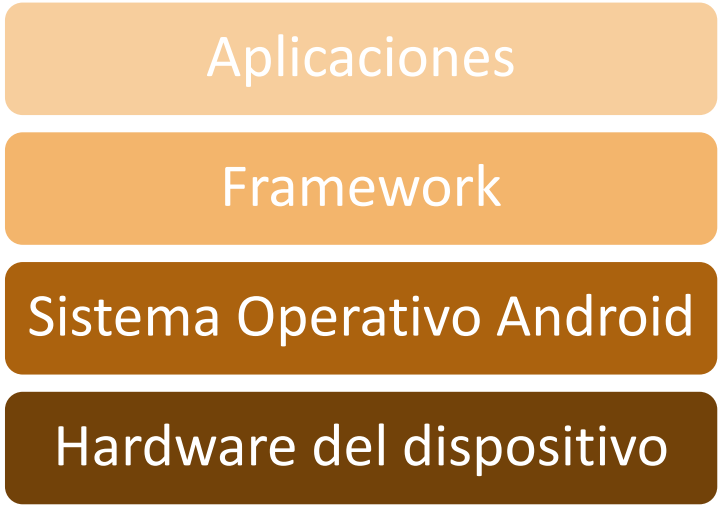
La forma de acceder al hardware del dispositivo desde el *Framework* se hace usando abstracciones de capas.

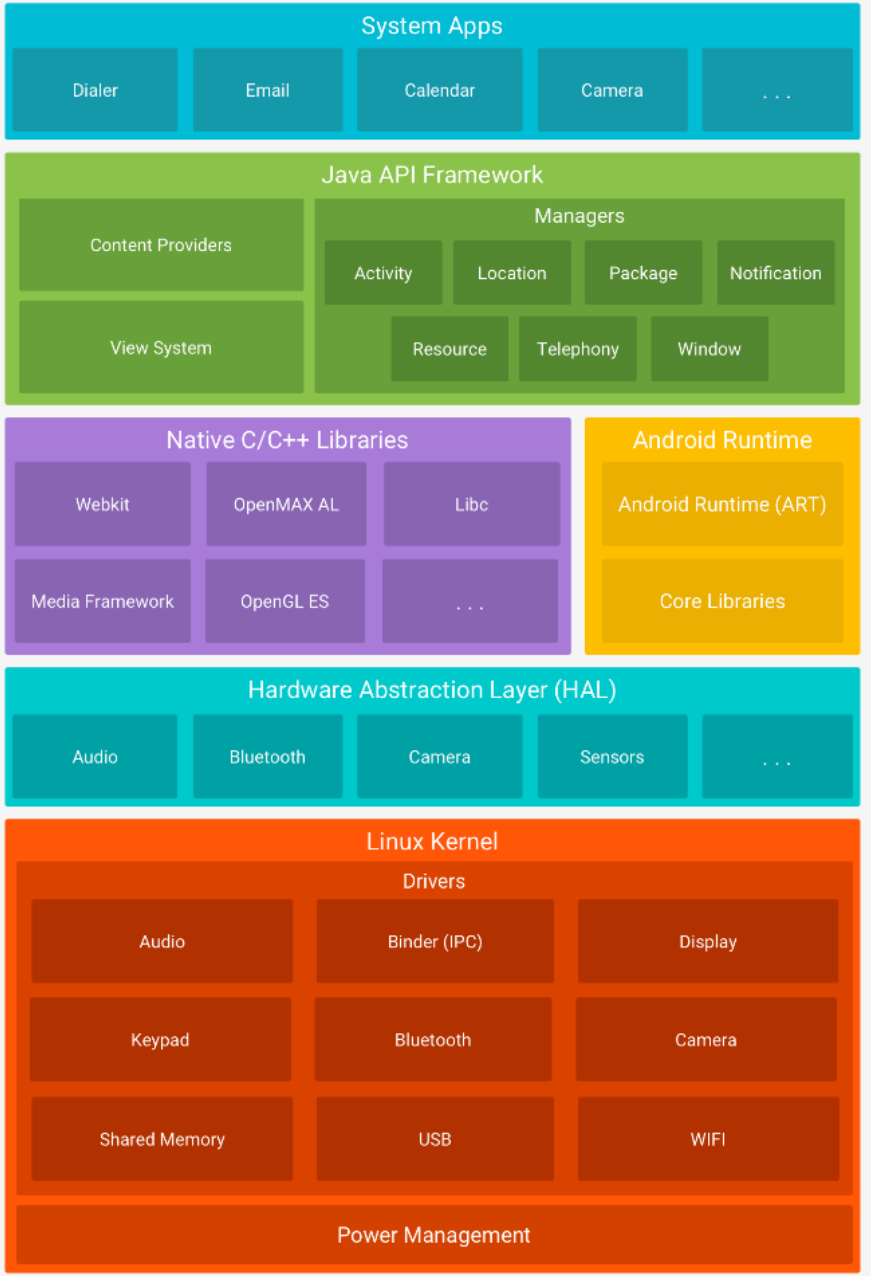
Cada capa se enfoca en resolver una parte del problema.

Las capas de más abajo se centran en resolver problemas más pequeños y finalmente con todas las capas combinadas se obtiene la funcionalidad requerida.

Android: Arquitectura

Un resumen de su arquitectura es el siguiente:





https://developer.android.com/guide/platform

Android: Aplicación

Una aplicación *Android* es una colección de componentes que trabajan entre ellos usando el *Framework Android*.

Existen 4 tipos de componentes:

- Actividades
- Servicios
- *Content Providers*
- *Broadcast Receivers*

Android conoce la existencia en la aplicación de cada componente porque se registran en el fichero ***Android Manifest***.

Por ahora nos vamos a centrar en las **Actividades**.

Android: Actividad

Una **Actividad** es el componente *Android* responsable de prácticamente cualquier interacción de la aplicación con el usuario.

La documentación de Android la define como: “un componente de la aplicación que contiene una pantalla con la que los usuarios pueden interactuar para realizar una acción, como marcar un número telefónico, tomar una foto, enviar un correo electrónico o ver un mapa.“.

Las Actividades son las responsables de crear la ventana que la aplicación usa para dibujar y recibir los eventos del sistema.

Una aplicación típica contiene una serie de Actividades donde una de ellas es la primera en ejecutarse al lanzar (*launcher*) la ejecución de la aplicación.

Android: *Android Manifest*

El fichero ***AndroidManifest.xml*** contiene información esencial del proyecto como sus componentes o las librerías que necesita.

AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="My Application"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="My Application"
            android:theme="@style/Theme.MyApplication"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Android: *Android Manifest*

Dentro de la etiqueta ***application*** con algunas características de nuestra aplicación se encuentra la etiqueta para indicar el componente Actividad (***activity***) y dentro de este la etiqueta ***intent-filter*** que a su vez contiene otra etiqueta (***category***) que menciona ***LAUNCHER***.

AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="My Application"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="My Application"
            android:theme="@style/Theme.MyApplication"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Android Studio

Android Studio es el IDE oficial de *Android*.

Está diseñado para que *Android* pueda acelerar el desarrollo y nos permita crear las apps de mejor calidad para todos los dispositivos de *Android*.

Ofrece herramientas personalizadas para programadores de *Android*.

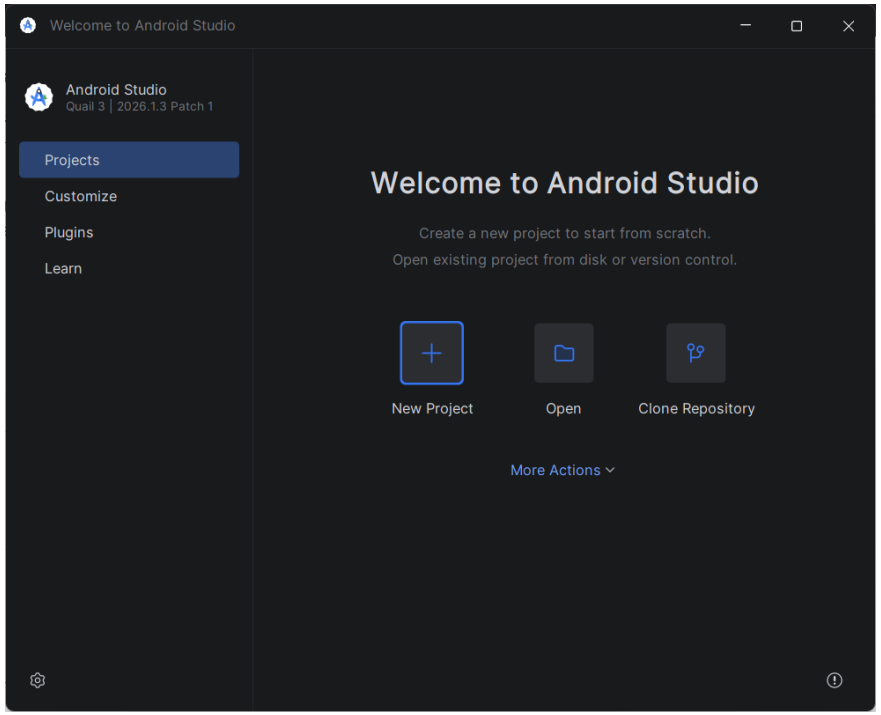
Se incluyen herramientas completas de edición, depuración, pruebas y perfilamiento de códigos.

Página oficial: https://developer.android.com/studio

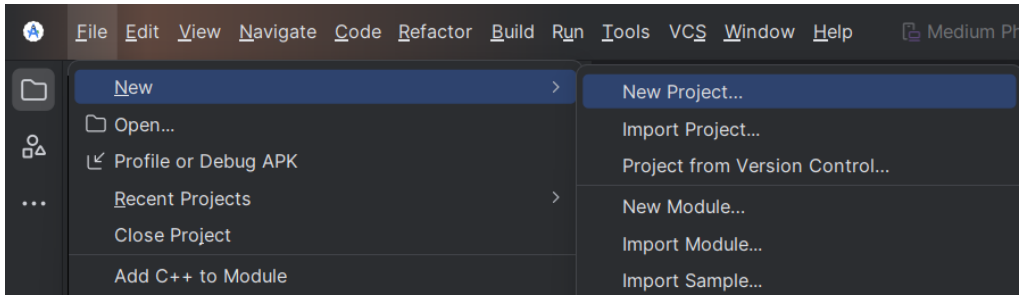
Guía de su instalación y uso: https://developer.android.com/studio/install?hl=es-419

Android Studio: Crear nuevo Proyecto

Si no tenemos abierto un proyecto en Android Studio:

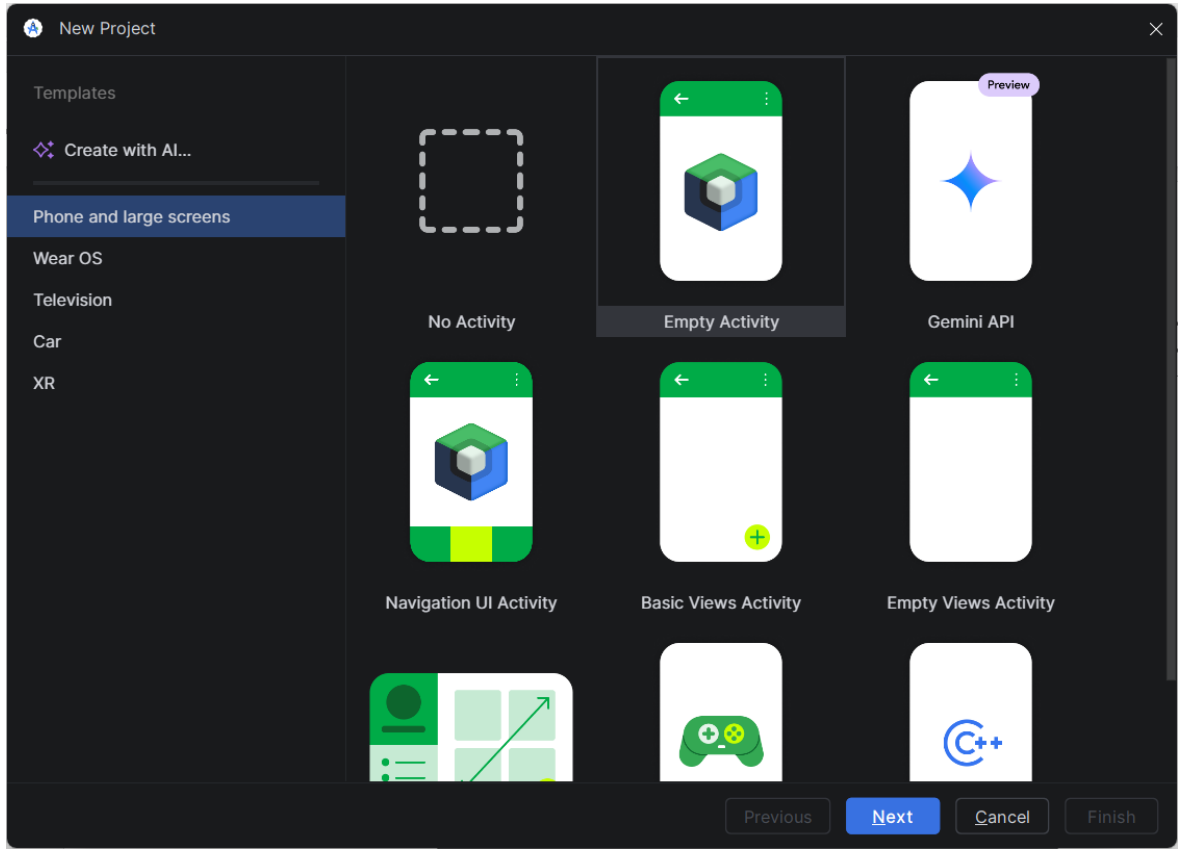


Si ya tenemos un proyecto abierto:



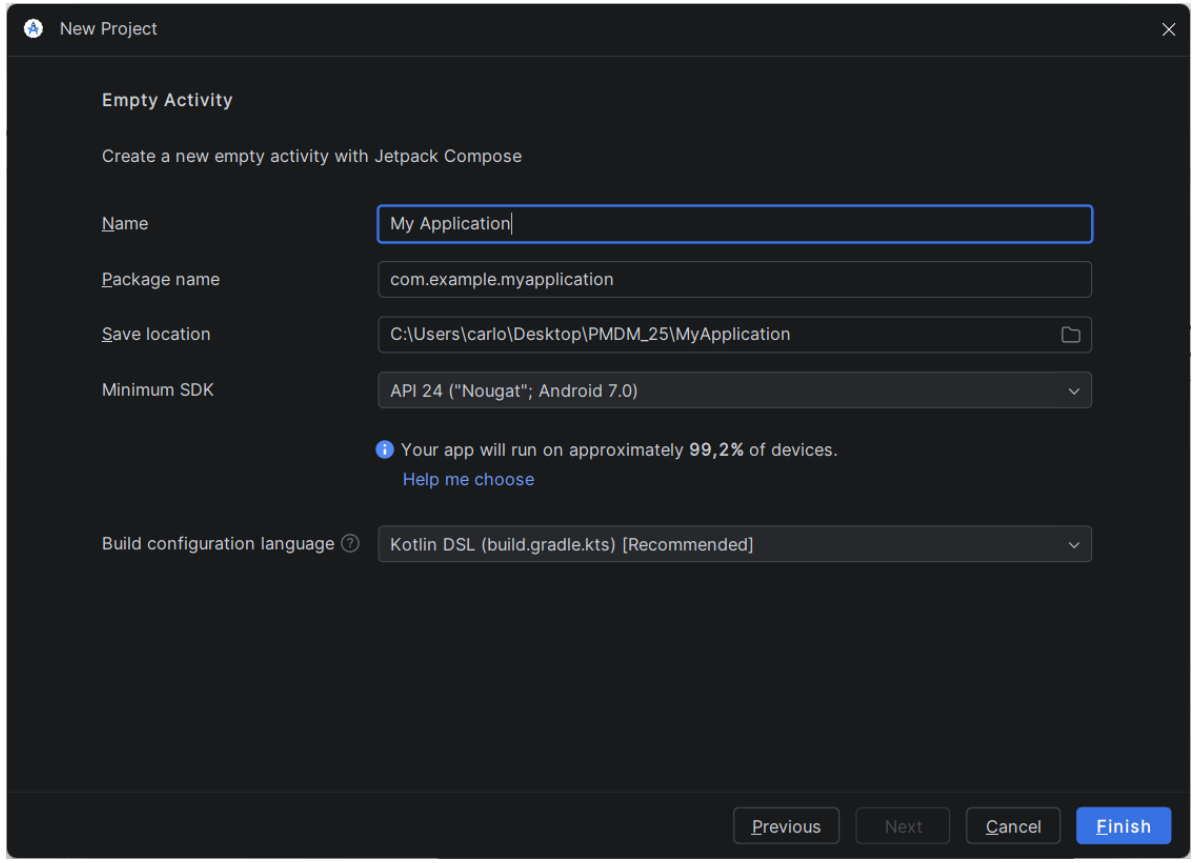
Android Studio: Crear nuevo Proyecto

Seleccionamos el tipo de proyecto:

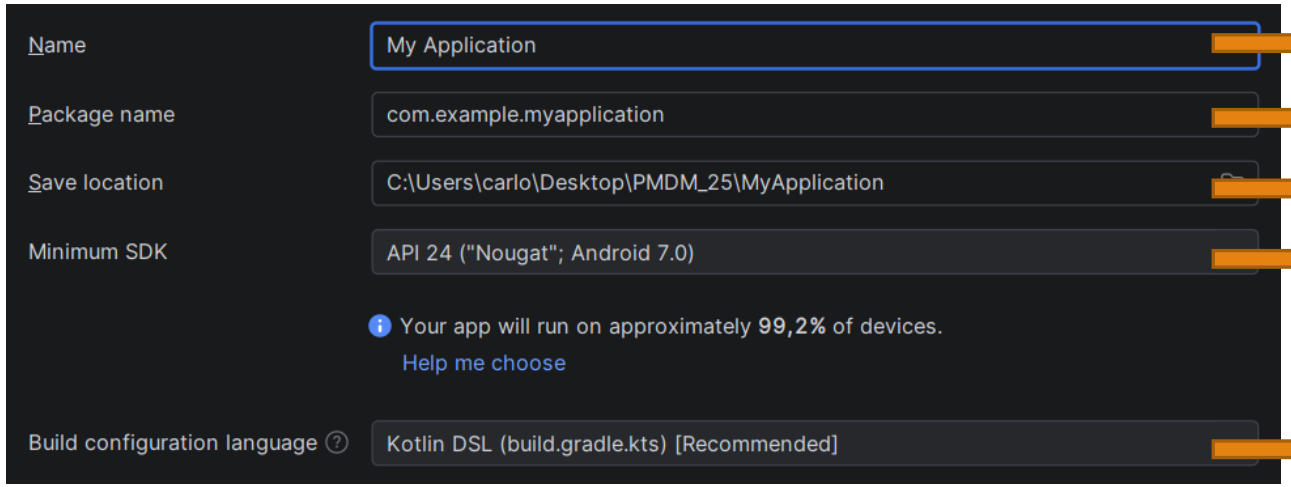


Android Studio: Crear nuevo Proyecto

Configuramos el proyecto:



Android Studio: Crear nuevo Proyecto



Nombre de la aplicación

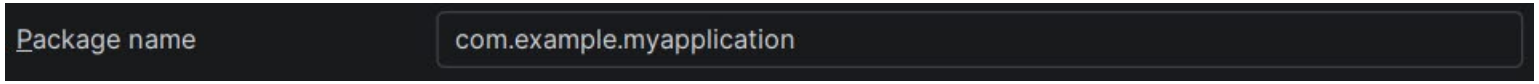
Nombre del paquete.

Ruta del proyecto.

Versión de API mínima que usará el proyecto.

Lenguaje de programación que vamos a usar.

Android Studio: Crear nuevo Proyecto



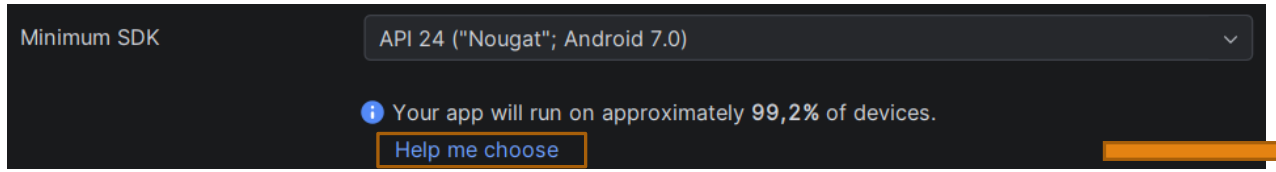
Es importante poner un nombre de paquete único si tenemos pensado subir la aplicación en ***Google Play*** ya que este debe ser único.

El nombre del paquete está formado por el dominio de la empresa invertido seguido del nombre de la aplicación sin espacios.

Por ejemplo: **com.carlosc.appTest**

- Siendo **carlosc.com** el dominio y **app Test** el nombre de la aplicación.
- Si compro ese dominio tendré el control total de las aplicaciones que empiecen con él.

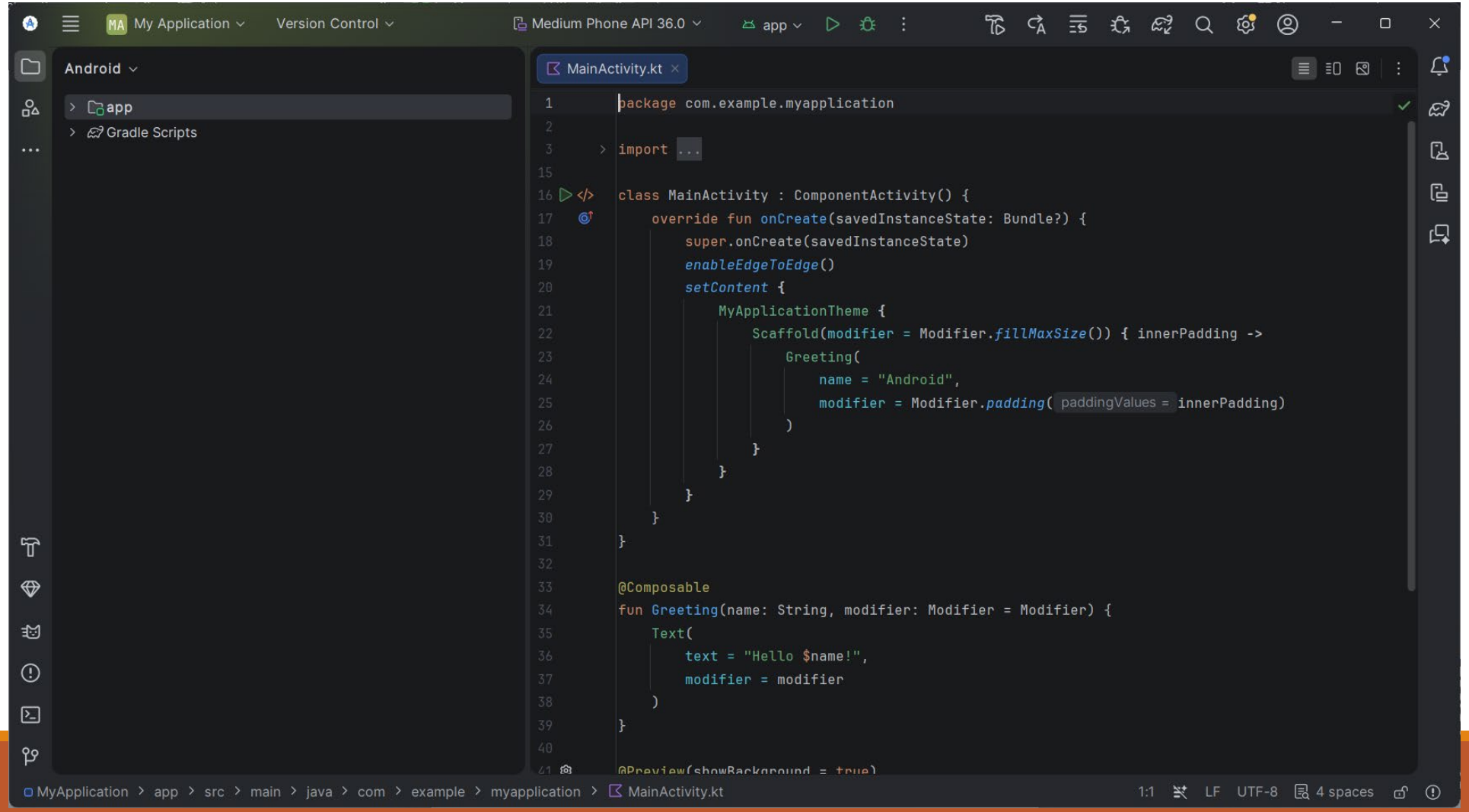
Android Studio: Crear nuevo Proyecto



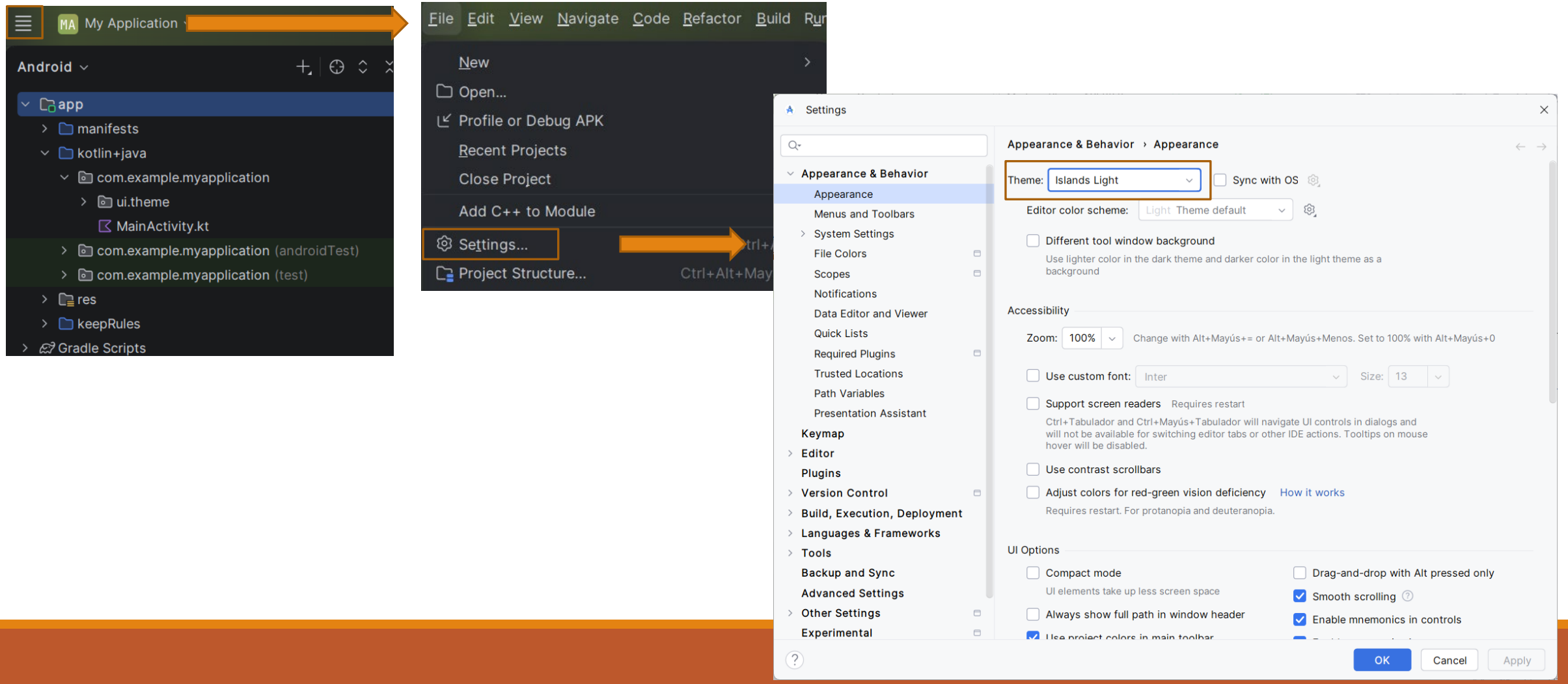
	ANDROID PLATFORM VERSION	API LEVEL	CUMULATIVE DISTRIBUTION
5	Lollipop	21	
5.1	Lollipop	22	99,9%
6	Marshmallow	23	99,6%
7	Nougat	24	99,2%
7.1	Nougat	25	98,8%
8	Oreo	26	98,4%
8.1	Oreo	27	97,6%
9	Pie	28	95,3%
10	Q	29	90,8%
11	R	30	83,0%
12	S	31	69,3%
13	T	33	57,9%
14	U	34	44,0%
15	V	35	26,8%
16	B	36	7,5%

Last updated: December 1, 2025

Android Studio: IU

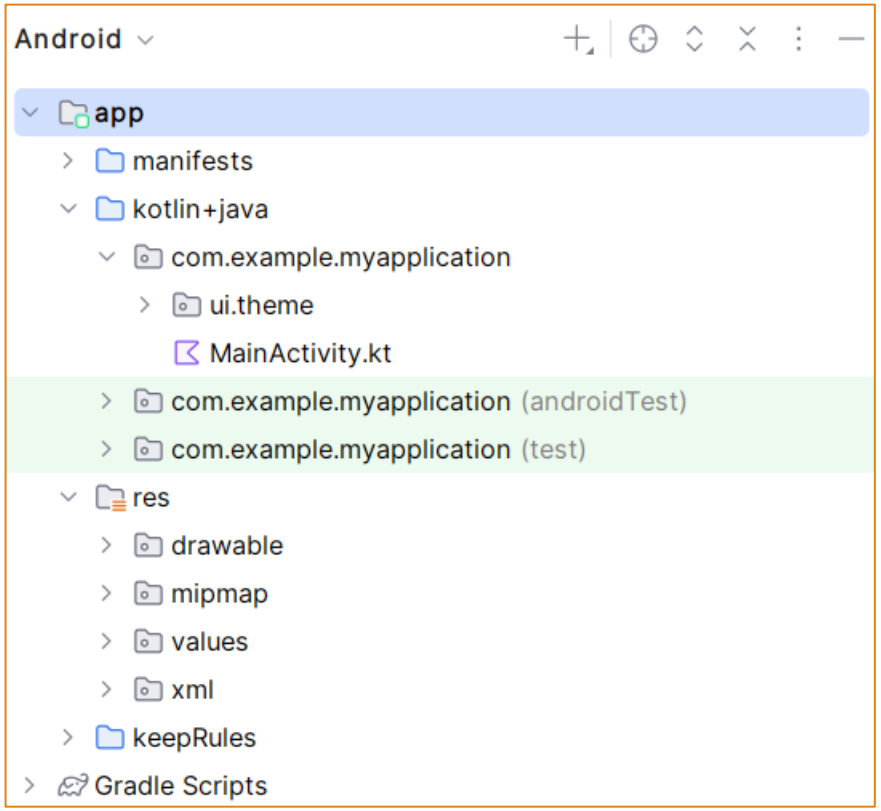


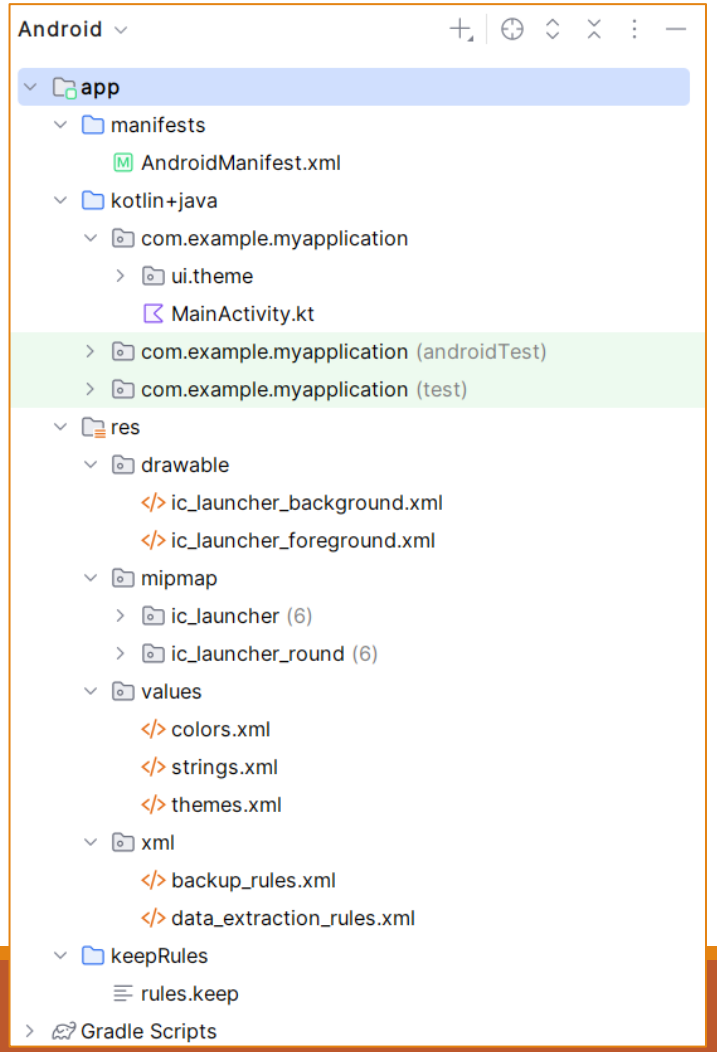
Android Studio: IU (Cambio de tema)



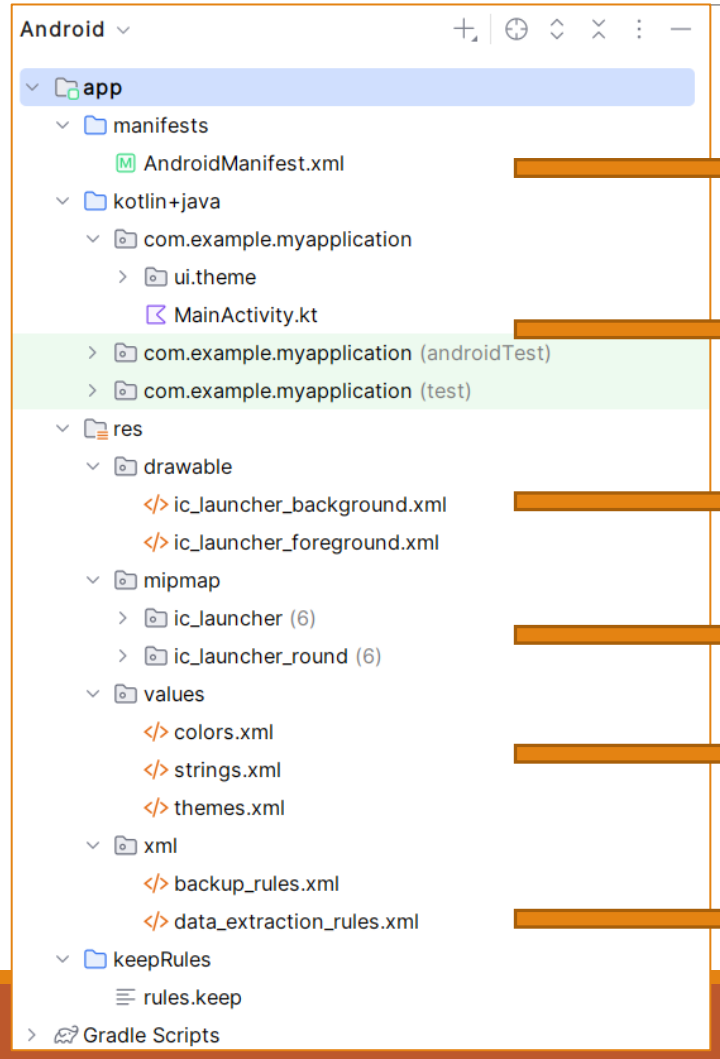
Android Studio: Ficheros del Proyecto

Los ficheros creados son:





Android Studio: Ficheros del Proyecto



Contiene información esencial del proyecto como sus componentes o las librerías que necesita.

Actividad principal. Cada actividad indica el comportamiento de la aplicación con el usuario.

Contiene las imágenes utilizadas en la aplicación.

Contiene los iconos de la aplicación.

colors.xml: Contiene los colores utilizados en la aplicación.
strings.xml: Contiene las cadenas de texto utilizadas en la aplicación (nombre de la aplicación, texto de los botones, etc.).
themes.xml: Contiene los temas y estilos utilizados en la aplicación.

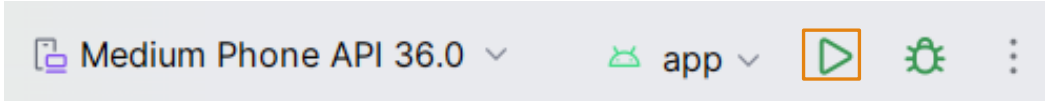
Contiene información como las reglas para realizar copias de seguridad o preferencias de la aplicación.

Android Studio: Ejecutar proyecto

Existen dos formas de ejecutar nuestra aplicación en *Android Studio*:

1. Desde el móvil.
2. Desde el emulador.
   - *Android Emulator* simula un dispositivo con el que puedes interactuar desde el ordenador.
   - Permite instalar y probar aplicaciones sin usar un dispositivo real.
   - Es compatible con teléfonos y tablets *Android*, y con dispositivos *Android Wear* y *Android TV*.

En ambos casos hemos de pulsar sobre el icono:



Android Studio: Ejecutar proyecto

Para ejecutar una aplicación en **el móvil**:

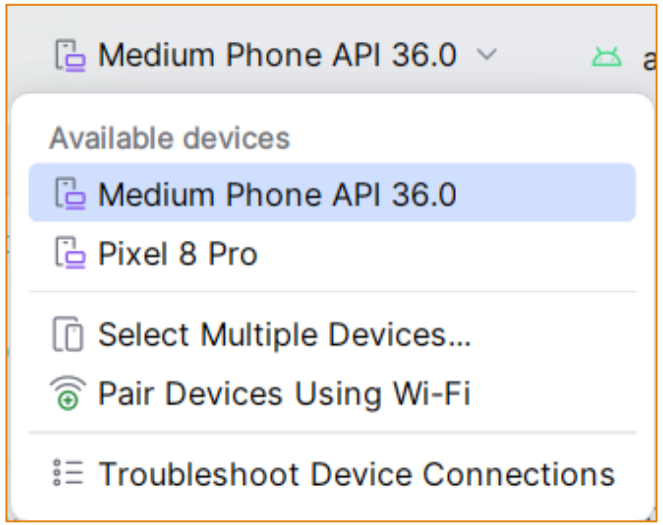
1. Hay que indicarle que somos programadores. Para eso hay que ir a ***Ajustes > Sobre el teléfono > Número de compilación*** (o similar) y **pulsar 7 veces** sobre el botón y aparecerá la opción ***Opciones del programador***. Entramos y activamos la opción ***Depuración por USB***.
2. Conectar el móvil al ordenador mediante el cable USB.
3. Si estás en *Windows* necesitas un *driver* para la comunicación. https://developer.android.com/studio/run/oem-usb

   Una vez descargado, se debe conectar el móvil y hacer clic en el **Gestor de dispositivos** y en otros dispositivos buscaremos nuestro móvil o ***ABD interface*** (*Android Debug Bridge*). Por último pulsaremos el botón derecho del ratón y seleccionaremos actualizar *driver software* y seleccionaremos el *driver*.

Android Studio: Ejecutar proyecto

Para ejecutar una aplicación en **el emulador**:

Si ya hemos creado nuestro **AVD** (*Available Virtual Device*) lo seleccionamos y lo arrancamos:

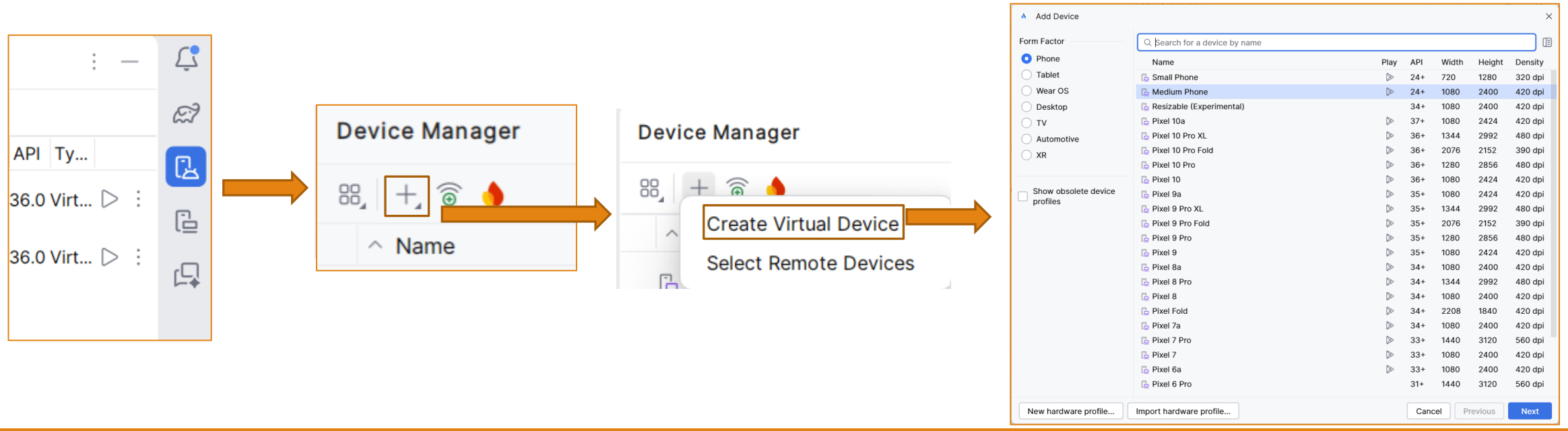


Android Studio: Ejecutar proyecto

Para ejecutar una aplicación en **el emulador**:

Sino está creado o queremos crear uno nuevo:

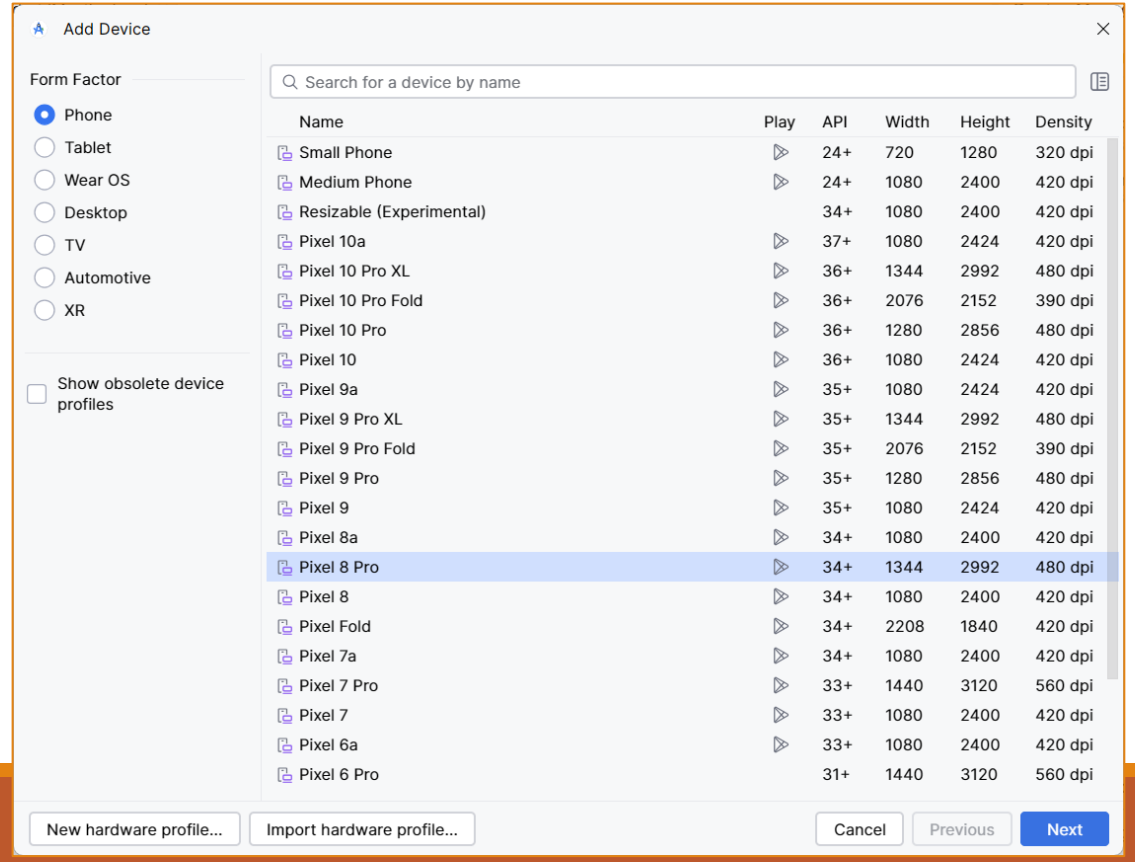
1. Pulsamos el botón ***Device Manager*** y después ***Add a new device > Create Virtual Device***:



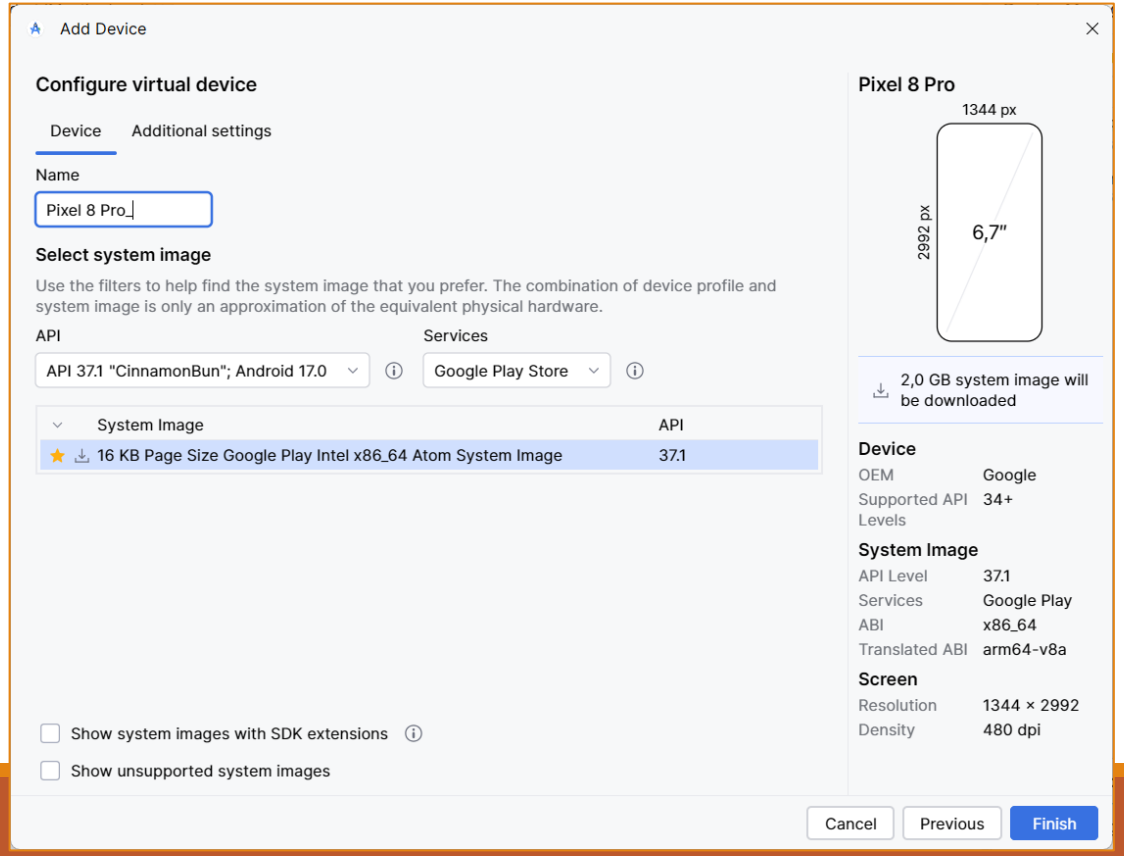
Android Studio: Ejecutar proyecto

Para ejecutar una aplicación en **el emulador**:

2. Elegimos el dispositivo a emular:



3. Y descargamos la imagen del sistema a utilizar:



Android Studio: Ejecutar proyecto

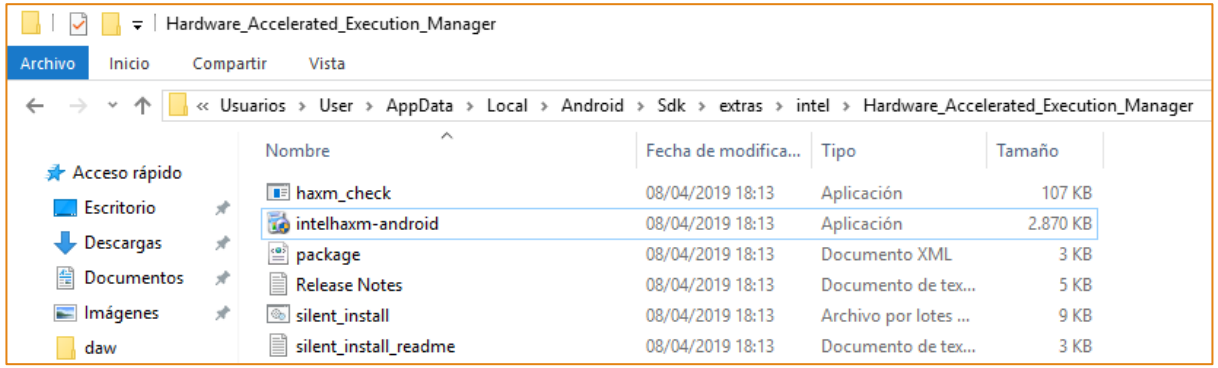
Para ejecutar una aplicación en **el emulador**:

4. Pulsamos sobre ***Next*** y por último en ***Finish***.
5. Activar la virtualización ***VTx*** desde la BIOS.
6. Asegurarnos que el ***x86 Emulator Accelerator*** (*HAXM installer*) está instalado. Para eso hemos de ir a ***Tools > SDK Manager*** y ver que la opción está seleccionada.

Android Studio: Ejecutar proyecto

Para ejecutar una aplicación en **el emulador**:

7. Si estamos en Windows, deberemos ir a ***C:\Users\User\AppData\Local\Android\Sdk\extras\intel\Hardware_Accelerated_Execution_Manager*** y ejecutar el archivo ***intelhaxm-android.exe***. Este archivo es el que realmente lo instala.



Reiniciar Android Studio.

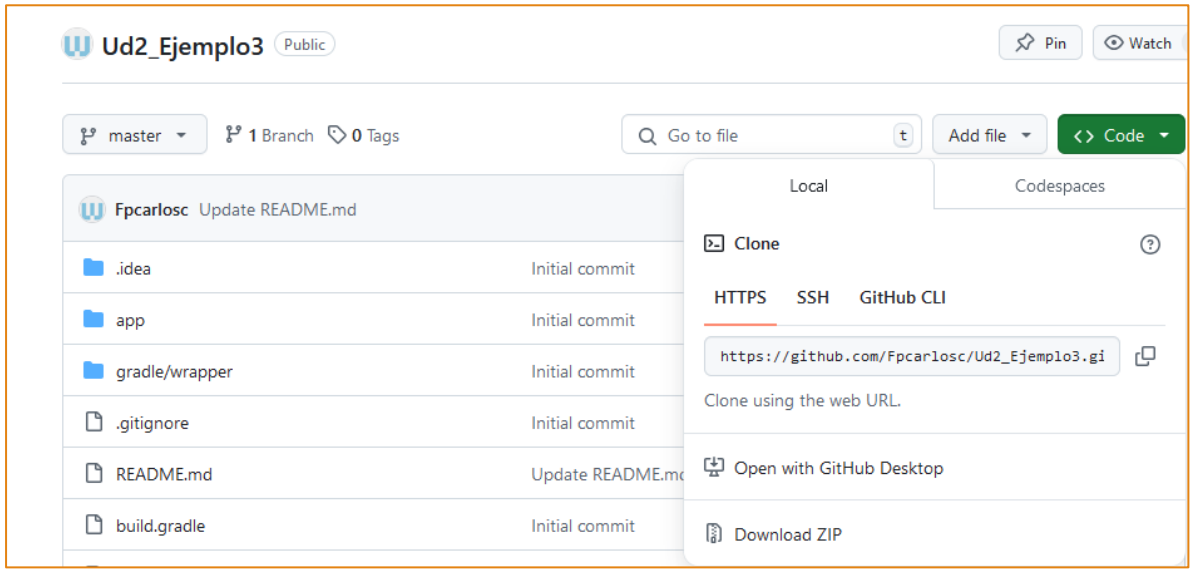
Android Studio: Ejecutar proyecto



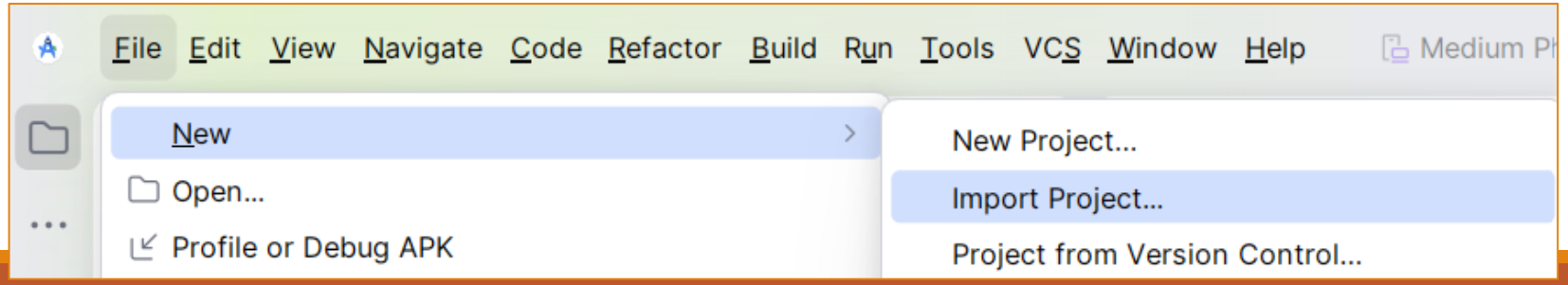
Android Studio: Importar proyecto

Durante el curso vamos a importar proyectos desde *GitHub*. Para ello:

1. Lo descargamos desde *GitHub*:



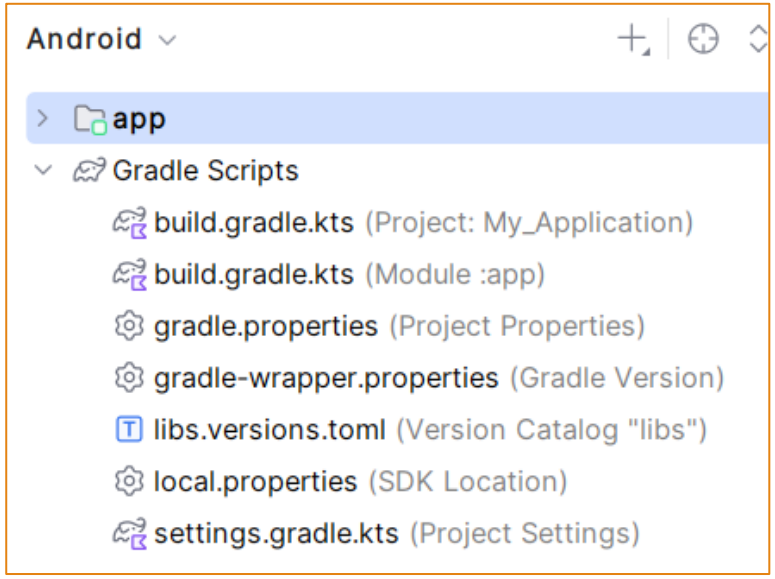
2. Lo descomprimimos.
3. Y lo importamos:



Android Studio: *Gradle*

Gradle es el sistema automatizado de construcción que utiliza *Android Studio*.

Cuando creamos una aplicación se generan automáticamente algunos scripts *Gradle*:



Android Studio: *Gradle*

Cuando importamos un proyecto podemos tener problemas de versiones del SDK, con lo que podemos actualizar las dependencias y versiones del proyecto desde los ficheros **libs.versions.toml** y ***build.gradle (Module:app)***:

libs.versions.toml

```
junit = "4.13.2"
junitVersion = "1.1.5"
espressoCore = "3.5.1"
lifecycleRuntimeKtx = "2.6.1"
activityCompose = "1.8.0"
kotlin = "2.2.10"
composeBom = "2026.02.01"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
```

build.gradle.kts (:app)

```
android {
    buildTypes {
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
    buildFeatures {
        compose = true
    }
}

dependencies {
    implementation(platform( dependencyProvider = libs.androidx.compose.bom))
    implementation(libs.androidx.activity.compose)
    implementation(libs.androidx.compose.material3)
    implementation(libs.androidx.compose.ui)
    implementation(libs.androidx.compose.ui.graphics)
    implementation(libs.androidx.compose.ui.tooling.preview)
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    testImplementation(libs.junit)
    androidTestImplementation(platform( dependencyProvider = libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.compose.ui.test.junit4)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(libs.androidx.junit)
    debugImplementation(libs.androidx.compose.ui.test.manifest)
    debugImplementation(libs.androidx.compose.ui.tooling)
}
```

Android Studio: *Debugger*

Podemos añadir puntos de interrupción clicando al lado de la línea en la que queremos que se pare la ejecución, donde aparecerá un punto rojo:

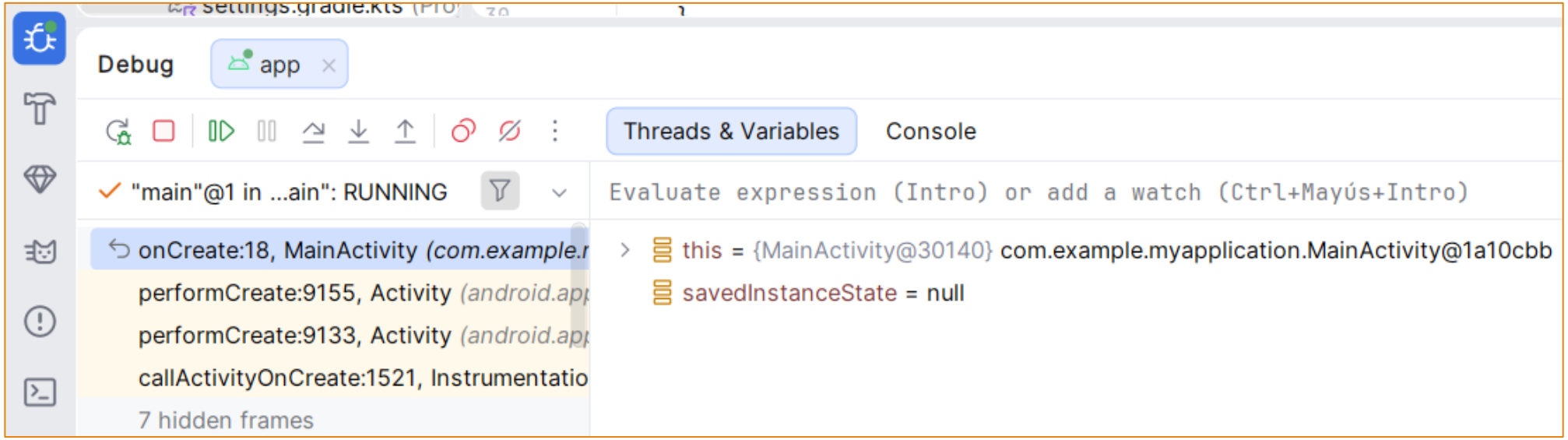


Para acceder al modo **debug** clicamos sobre el icono:



Android Studio: *Debugger*

Y podremos ver la consola de *debug*:



https://developer.android.com/studio/debug

Android Studio: Consola *Logcat*

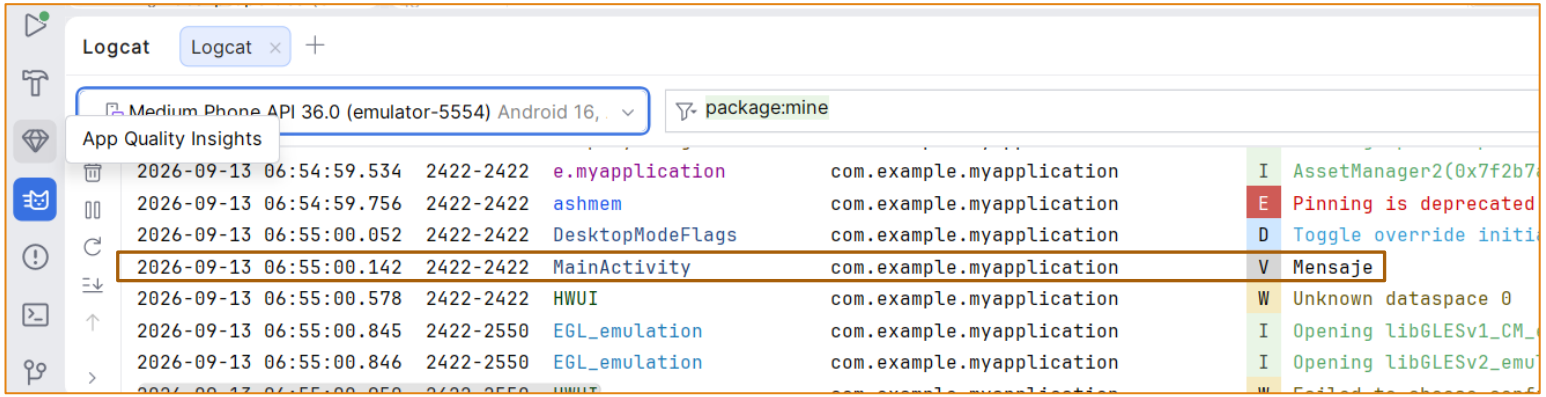
Android Studio dispone de una consola ***Logcat*** donde se puede ver todos los mensajes de ***Log***.

Podemos crear mensajes de *Log* de distinto tipo:

- e(String etiqueta, String mensaje) (error)
- w(String etiqueta, String mensaje) (warning)
- i(String etiqueta, String mensaje) (information)
- d(String etiqueta, String mensaje) (debug)
- v(String etiqueta, String mensaje) (verbose)

Android Studio: Consola *Logcat*

```
17  class MainActivity : ComponentActivity() {
18      companion object {
19          const val LOG_TAG:String = "MainActivity"
20      }
21
22      override fun onCreate(savedInstanceState: Bundle?) {
23          super.onCreate(savedInstanceState)
24          enableEdgeToEdge()
25
26          Log.v(LOG_TAG, msg = "Mensaje")
27
28          setContent {
29              MyApplicationTheme {
30                  Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
31                      Greeting(
32                          name = "Android",
33                          modifier = Modifier.padding( paddingValues = innerPadding)
34                      )
35                  }
36              }
37          }
38      }
39  }
```



Cuando hayamos descubierto el problema y veamos que todo funciona bien debemos quitar del código los *Logs* ya que la pila que los almacena tiene un tamaño limitado.

Si ponemos muchos *Logs* puede que un mensaje importante desaparezca pronto.